Meta-Metacognition Repository Summary

Repository: https://github.com/pazhenchira/meta-metacognition
Date: February 6, 2026

Overview

A **meta-cognitive orchestration engine** that builds complete production-ready applications from plain English descriptions by orchestrating multiple AI agents with different specialized roles.

Core Concept

The system acts as a "virtual CTO" that applies 50+ years of engineering wisdom (Thompson, Knuth, Pike, Kernighan) to systematically build applications.

Key Innovation: The **"source code" is actually AI agent prompts** (Markdown files in `.meta/`) rather than traditional code.

How Agent Orchestration Works

1. Hierarchical Role Structure (`.meta/roles/`)

The system has **13 specialized AI agent roles**, each with clear responsibilities:

- **Strategy Owner** - Defines business goals and success metrics
- **Product Manager** - Translates requirements into features
- **Architect** - Designs system architecture using LEGO principles (single-responsibility components)
- **Designer** - Creates UX/UI with usability principles (Krug, Norman)
- **Developer** - Implements code following KISS principles
- **Tester** - Writes comprehensive tests (>80% coverage)
- **Operations** - Handles deployment and infrastructure
- **Tech Writer** - Creates documentation
- **Essence Analyst** - Validates the app delivers its promised value
- **GTM roles** (Evangelist, Growth Marketer, Monetization Strategist) - Go-to-market strategy

2. Workflow-Based Orchestration (`.meta/workflows/`)

The engine automatically selects and coordinates roles based on the work type:

- **new_feature.md** - When building a new feature:
  - Strategy Owner → PM → Architect → Designer → Developer → Tester → Operations
- **bug_fix.md** - For bugs:
  - Operations/Dev first, PM optional
- **enhancement.md** - For improvements:
  - Similar flow with emphasis on Architect review
- **Review gates** - Each role must "sign off" before the next role begins

3. Automatic Role Assignment

The **Meta-Orchestrator** (`.meta/AGENTS.md`) acts as the conductor:

1. **User describes app** in `app_intent.md` (plain English)

2. **Meta-Orchestrator analyzes** the intent and determines:
    - Which roles are needed (e.g., trading app needs Architect for risk management)
    - What order to execute them (dependency graph)
    - Which can work in **parallel** (session isolation)
3. **Spawns specialized agents** - Each role gets:
    - Its own context/session (prevents context collapse)
    - Specific instructions from `.meta/roles/{role}.md`
    - Access to wisdom files (engineering best practices)
    - Templates for artifacts (`.meta/templates/`)
4. **Coordinates through review gates** - Each agent's output is reviewed before proceeding

4. LEGO Decomposition (Thompson #5: "Do one thing well")

The Architect role automatically breaks complex apps into **single-responsibility components**:

```
Complex App → LEGO Components
├── Auth LEGO (handles authentication only)
├── Data LEGO (handles data operations only)
├── UI LEGO (handles interface only)
└── API LEGO (handles external communication only)
```

Each LEGO gets its own orchestrator that spawns: Designer → Developer → Tester (parallel work streams).

5. Session Isolation (Prevents Context Collapse)

Unlike traditional AI that runs in one conversation, this system:

- **Spawns independent agents** for each component (like a real team)
- Each agent has **focused responsibility** (no context overload)
- Coordinates through **file-based state** (`orchestrator_state.json` , `lego_state_*.json`)
- Can build **50+ component apps** without breaking

Key Innovation: Wisdom-Driven Decisions

Rather than rigid rules, the system uses **24,000+ lines of curated engineering wisdom**:

- **`.meta/wisdom/`** - Thompson's Unix philosophy, Knuth on optimization, Pike on simplicity
- **`.meta/patterns/`** - Detects antipatterns (God Object, Golden Hammer), suggests success patterns (Circuit Breaker)
- **Guides role decisions** - e.g., "Is this too complex? Apply KISS"

Example Orchestration Flow for New App

```
1. User: "Build a trading bot"

2. Meta-Orchestrator reads intent

3. Spawns Strategy Owner: Defines success = positive Sharpe ratio

4. Spawns Essence Analyst: Discovers core value = risk-adjusted returns
```

```
5. Spawns Product Manager: Creates feature list (order entry, risk limits)

6. Spawns Architect: Designs LEGO components:
   ├── Market Data LEGO
   ├── Strategy Engine LEGO
   ├── Risk Manager LEGO
   └── Order Executor LEGO

7. For EACH LEGO in parallel:
   ├── Spawns Designer: UX for that component
   ├── Spawns Developer: Implements code
   └── Spawns Tester: >80% test coverage

8. Integration & System Tests

9. Operations: Deployment guide

10. Tech Writer: Documentation

11. Essence validation: Does it achieve positive Sharpe ratio?
```

Two AGENTS.md Files - Critical Distinction

Root `/AGENTS.md` - "Maintenance Orchestrator"

Purpose: Instructions for **maintaining and improving the engine itself**

- **Role**: "Meta-Orchestrator Maintenance Agent" - like a DevOps engineer for the engine
- **Audience**: AI agent working on the meta-orchestrator codebase
- **Scope**: Modifying engine files in `.meta/`, adding features to the orchestration system
- **Key phrase**: "You are the MAINTENANCE ORCHESTRATOR for the meta-orchestrator engine itself"
- **Think of it as**: The blueprint for **evolving the tool** that builds apps

What it does:

- Guides how to add new features to the engine (e.g., "add support for new programming language")
- Defines how to update wisdom files, patterns, templates
- Ensures changes follow the engine's own principles (dogfooding - the engine applies KISS/LEGO to itself)
- Handles version updates, CHANGELOG maintenance

`.meta/AGENTS.md` - "Meta-Orchestrator" (Core Logic)

Purpose: Instructions for **building applications using the engine**

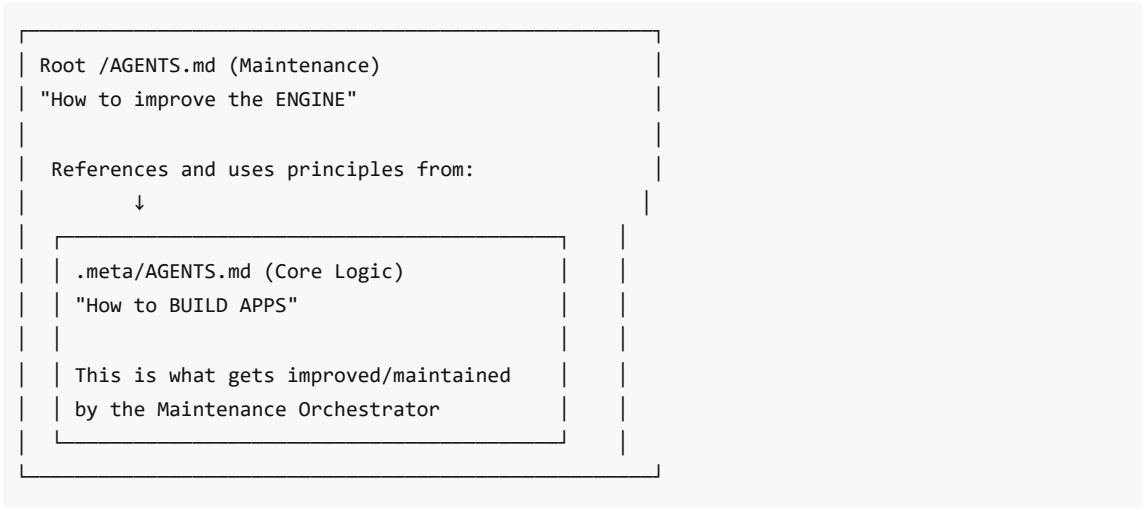
- **Role**: "Meta-Orchestrator" - like a virtual CTO building user apps
- **Audience**: AI agent building apps for end users
- **Scope**: Reading `app_intent.md`, decomposing into LEGOs, spawning specialized role agents
- **Key phrase**: "You are the META-ORCHESTRATOR for this repository" (meaning the user's app repo)
- **Think of it as**: The **operating manual** for the app-building tool

What it does:

- Contains the 12-phase pipeline (essence discovery → LEGO planning → implementation → validation)
- Defines how to spawn role agents (Architect, Developer, Tester, etc.)

- Implements session isolation, wisdom application, antipattern detection
- Orchestrates building complete apps from plain English

The Relationship (Recursive/Self-Referential)



The engine uses itself to improve itself (dogfooding):

- Root AGENTS.md says: "When improving the engine, use `.meta/AGENTS.md` as the source of truth"
- Root AGENTS.md applies the same principles that .meta/AGENTS.md applies to user apps

Practical Example

If you want to BUILD A TRADING BOT:

- Activate: `.meta/AGENTS.md`
- It reads your `app_intent.md`, spawns Architect/Developer/Tester roles, builds the bot

If you want to ADD A NEW FEATURE TO THE ENGINE (e.g., "support Rust apps"):

- Activate: `/AGENTS.md` (root)
- It modifies `.meta/AGENTS.md`, adds Rust templates, updates wisdom files

Why This Matters

This is **the only AI system** that:

1. **Automatically assigns specialized roles** based on work type
2. **Validates apps deliver their promised value** (essence validation)
3. **Generates self-documenting apps** with `AGENTS.md` for future maintenance
4. **Supports intelligent maintenance** (KEEP/REFACTOR/REGENERATE decisions)
5. **Prevents context collapse** through session isolation (can build 50+ component apps)
6. **Applies 50+ years of engineering wisdom** systematically (Thompson, Knuth, Pike, Kernighan)

Competitive Advantages

Feature	ChatGPT/Claude	GitHub Copilot	Cursor	Bolt.new	**Meta-Orchestrator**

Complete apps	❌ Snippets	❌ Autocomplete	⚠️ Single file	⚠️ Web apps	✅ Any app type
Architecture	❌ None	❌ None	❌ None	⚠️ Basic	✅ LEGO decomposition
Wisdom applied	❌ None	❌ None	❌ None	❌ None	✅ Thompson, Knuth, etc.
Tests	❌ Manual	❌ Manual	❌ Manual	⚠️ Basic	✅ >80% coverage
Essence validation	❌ None	❌ None	❌ None	❌ None	✅ End-to-end testing
Maintainability	❌ Throwaway	❌ Throwaway	⚠️ Limited	⚠️ Limited	✅ Self-documenting
Context limits	❌ Breaks at 20K	❌ Single file	⚠️ Single project	⚠️ Web only	✅ Session isolation

---

Key Repository Structure

```
meta-metacognition/
├── .meta/                          # ENGINE (core orchestration logic)
│   ├── AGENTS.md                   # Meta-orchestrator build logic (12 phases)
│   ├── roles/                      # 13 specialized agent roles
│   │   ├── architect.md
│   │   ├── developer.md
│   │   ├── tester.md
│   │   └── ... (10 more)
│   ├── workflows/                  # Work type routing
│   │   ├── new_feature.md
│   │   ├── bug_fix.md
│   │   └── enhancement.md
│   ├── wisdom/                     # 24,000+ lines of engineering principles
│   │   ├── engineering_wisdom.md   # Thompson, Knuth, Pike, Kernighan
│   │   ├── strategic_wisdom.md
│   │   ├── design_wisdom.md
│   │   └── risk_wisdom.md
│   ├── patterns/                   # Antipatterns and success patterns
│   │   ├── antipatterns.md         # God Object, Golden Hammer, etc.
│   │   └── success_patterns.md     # Circuit Breaker, Config Validator, etc.
│   └── templates/                  # Templates for generated artifacts
│       ├── AGENTS.template.md      # App-specific orchestrator
│       └── ...
├── AGENTS.md                       # Maintenance orchestrator (improves engine)
├── app_intent.md                   # User's app description (plain English)
├── essence.md                      # What makes the engine valuable
├── README.md                       # Human-centric documentation
└── ... (documentation files)
```

Success Metrics (For Generated Apps)

1. **KISS Compliance**: 100% single-responsibility principle
2. **Zero Antipatterns**: No God Objects, Golden Hammers, Magic Numbers
3. **Test Coverage**: >80% on all generated code
4. **Essence Delivery**: 100% pass end-to-end validation
5. **Maintainability**: 100% include AGENTS.md for future work
6. **Time-to-First-Value**: 15-45 minutes from idea to working app

---

How to Use It

1. **Clone the repo**: `git clone https://github.com/pazhenchira/meta-metacognition`
2. **Edit** `app_intent.md` : Describe your app in plain English
3. **Run the meta-orchestrator**: `@workspace Act as meta-orchestrator in .meta/AGENTS.md and build this app`
4. **Answer 2-3 questions**: Engine asks clarifying questions
5. **Approve LEGO plan**: Review architecture before building
6. **Get complete app**: Working code, tests, docs, deployment guide in 15-45 min

---

References

- **Repository**: https://github.com/pazhenchira/meta-metacognition
- **Version**: 2.0.34 (January 9, 2026)
- **License**: MIT
- **Key Docs**:
  - README.md - Getting started
  - essence.md - Why this exists
  - AGENTS.md - Maintenance instructions
  - .meta/AGENTS.md - Build orchestration logic
  - .meta/roles/ - Agent role definitions

Summary generated: February 6, 2026